Software Requirements Specification

for

Local Tourist Day-Visit Planner and Information System

Version 1.0 approved

Prepared by Witharanage WAH (E2529013)

Institute of Technology, Bachelor of IT (BIT)

ITE2953 - Individual Project

August 21, 2026

Table of Contents

Table of Contents 2
Revision History 4
1. Introduction 5
1.1 Purpose 5
1.2 Document Conventions 5
1.3 Intended Audience and Reading Suggestions 5
1.4 Product Scope 5
1.5 References 5
2. Overall Description 7
2.1 Product Perspective 7
2.2 Product Functions 7
2.3 User Classes and Characteristics 7
2.4 Operating Environment 8
2.5 Design and Implementation Constraints 9
2.6 User Documentation 9
2.7 Assumptions and Dependencies 9
3. External Interface Requirements 11
3.1 User Interfaces 11
3.2 Hardware Interfaces 11
3.3 Software Interfaces 11
3.4 Communications Interfaces 12
4. System Features 13
4.1 Place Directory with Category Filtering 13
4.1.1 Description and Priority 13
4.1.2 Stimulus/Response Sequences 13
4.1.3 Functional Requirements 13
4.2 Detailed Place View with Map Integration 13
4.2.1 Description and Priority 13
4.2.2 Stimulus/Response Sequences 14
4.2.3 Functional Requirements 14
4.3 Day-Visit Itinerary Builder 14
4.3.1 Description and Priority 14
4.3.2 Stimulus/Response Sequences 14
4.3.3 Functional Requirements 14
4.4 Administrator Content Management (CRUD) 15
4.4.1 Description and Priority 15

4.4.2 Stimulus/Response Sequences 15
4.4.3 Functional Requirements 15
4.5 Administrator Authentication and User Management 16
4.5.1 Description and Priority 16
4.5.2 Stimulus/Response Sequences 16
4.5.3 Functional Requirements 16
5. Other Nonfunctional Requirements 17
5.1 Performance Requirements 17
5.2 Safety Requirements 17
5.3 Security Requirements 17
5.4 Software Quality Attributes 18
5.5 Business Rules 18
6. Other Requirements 19
Appendix A: Glossary 20
Appendix B: Analysis Models 21
Appendix C: To Be Determined List 22

Revision History

Name	Date	Reason For Changes	Version
Witharanage WAH	21-Aug-2026	Initial draft of the SRS based on the approved project proposal	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Local Tourist Day-Visit Planner and Information System, Version 1.0. The system is a web-based application intended to help tourists discover places of interest within a 25 km radius of Walasmulla, Southern Province, Sri Lanka, and to plan a simple one-day visit itinerary. This document covers the complete scope of the system, including both the tourist-facing web application and the administrator content management dashboard.

1.2 Document Conventions

This document follows the IEEE-recommended SRS structure, adapted from the Karl E. Wiegers SRS template. Functional requirements are uniquely identified using the format REQ-<Feature Number>.<Sequence Number> (for example, REQ-1.1). Unless otherwise stated, all requirements within a feature are considered to be of equal priority to that feature's stated priority level. Headings are numbered hierarchically, and defined terms are capitalised and listed in Appendix A: Glossary.

1.3 Intended Audience and Reading Suggestions

This document is intended for the project supervisor/evaluator, the student developer (acting as both analyst and implementer), and any peer reviewers assessing the project. Readers who wish to understand the overall purpose and context of the system should read Sections 1 and 2. Readers interested in the interface and functional details should proceed to Sections 3 and 4. Section 5 is intended for readers evaluating quality, security, and performance considerations, and Section 6 together with the appendices provides supporting reference material.

1.4 Product Scope

The system will utilize the MERN stack (MongoDB, Express.js, React.js, and Node.js) to create the small-scale, realistic tourism information and visit-planning online application known as the Local Tourist Day-Visit Planner and Information System.. Tourists can browse, search, and filter locations by category; view comprehensive information such as opening hours, travel advice, and

map locations; and create a straightforward one-day visit plan by choosing locations thanks to the system, which centralises previously dispersed information about at least ten local places of interest around Walasmulla. Authorised personnel can manage registered tourist accounts and add, amend, and remove place records via an administrator dashboard. This system specifically excludes payment gateways, booking services, and hotel or transportation reservation systems.

1.5 References

[1] ITE2953, "Project Introduction and Problem Specification (1)," Institute of Technology, 2025.

[2] Meta Platforms, Inc., "React - A JavaScript library for building user interfaces," React.dev. [Online]. Available: https://react.dev/. [Accessed: Aug. 10, 2026].

[3] OpenJS Foundation, "Express - Node.js web application framework," Expressjs.com. [Online]. Available: https://expressjs.com/. [Accessed: Aug. 10, 2026].

[4] MongoDB, Inc., "MongoDB Documentation," Mongodb.com. [Online]. Available: https://www.mongodb.com/docs/. [Accessed: Aug. 10, 2026].

[5] Google Developers, "Maps JavaScript API," Google.com. [Online]. Available: https://developers.google.com/maps/documentation/javascript. [Accessed: Aug. 10, 2026].

[6] Sri Lanka Tourism Development Authority (SLTDA), "Official Corporate Website," sltda.gov.lk. [Online]. Available: https://www.sltda.gov.lk/. [Accessed: Aug. 10, 2026].

[7] K. E. Wiegers, "Software Requirements Specification Template," Process Impact, 1999.

2. Overall Description

2.1 Product Perspective

The Local Tourist Day-Visit Planner and Information System is not an extension of an already-existing product; rather, it is a brand-new, independent, three-tier online application. The system is divided into three parts: an Application Tier (a Node.js/Express.js backend that exposes a REST API through Authentication, Place Management, and Visit Plan Generator modules), a Client Tier (a React.js web browser front end serving both a Tourist Interface and an Administrator Interface), and a Data Tier (a MongoDB database storing place, category, and administrator account data). In order to show users the locations of places, the application additionally interfaces with an external Map API (such as Leaflet.js or Google Maps). This high-level design is theoretically replicated in Appendix B of this document and is depicted in Figure 1 of the accepted project proposal.

2.2 Product Functions

In general, the system will enable users to carry out the following significant tasks:

- **Browse and search** a category-filtered directory of nearby places of interest (nature, heritage, religious, cultural).
- **View comprehensive details** about each venue, such as its location on a map, distance, opening hours, travel advice, and description.
- **Automatically create** a straightforward one-day visit plan by choosing several locations, and register as a tourist to save or maintain this plan.
- **Add, update, and remove** place records by securely logging in as an administrator.
- **Manage** registered tourist accounts, including the option to block or delete accounts that abuse the system.

2.3 User Classes and Characteristics

The system identifies two primary user classes, summarized in Table 1.

User Class	Description	Key Characteristics
Tourist User	Local or foreign visitors who want to explore places of interest around Walasmulla and build a one-day visit plan.	No login required; varying levels of digital literacy; primarily accesses the system via mobile browsers; needs quick, clear information.
Administrator	The system manager responsible for maintaining accurate place information and managing registered tourist accounts.	Authenticated user with elevated privileges; comfortable with dashboard-style CRUD interfaces; uses a desktop browser.
Project Evaluator / Supervisor	Academic staff responsible for assessing the project and ensuring it meets the academic and technical standards.	Focuses on checking whether the final system accurately fulfills the requirements defined in this SRS document.
Local Site Managers (Indirect)	The authorities, caretakers, or management boards of the actual physical locations.	Expects accurate representation of their sites, including correct opening hours, rules, and respectful travel tips.
SLTDA / Local Authorities (Indirect)	Regional or national tourism governing bodies like the Sri Lanka Tourism Development Authority.	Provides the regulatory context; their guidelines and standards for tourism information indirectly influence how data should be presented.

Table 1: User Classes and Characteristics

2.4 Operating Environment

The system functions as a responsive web application that can be accessed on desktop, tablet, and mobile devices using contemporary web browsers like Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. The backend runs on a Node.js runtime environment with an Express.js server, and data is persisted in a MongoDB database, which may be hosted locally during development or on a cloud provider (such as MongoDB Atlas) for deployment. The client application requires an active internet connection to retrieve place data and map tiles from the external Map API.

2.5 Design and Implementation Constraints

- According to the approved project proposal, the system must be constructed utilising the MERN stack, which consists of MongoDB, Express.js, React.js, and Node.js.
- Development is limited to a 12-week academic project timeframe, which is broken down into phases for requirements, design, development, testing, and finalisation (see to Appendix B).
- To show place locations, the system must interface with an external map service (such as Leaflet.js or the Google Maps API); usage is governed by the terms of service and API limitations of that provider.
- The system does not include hotel or transport reservation integrations, booking services, or payment gateways because it is an individual academic project.
- Git must be used to maintain version control, and GitHub must be used to keep commit history throughout development.

2.6 User Documentation

The Tourist Interface will include tooltips and a brief on-screen help section to assist new users in creating a visit plan and filtering locations. The final project documentation will include a brief Administrator Guide that explains how to manage user accounts, log in, and carry out CRUD actions on place entries. Since this scholarly project is minor in scope, formal printed guides are not anticipated.

2.7 Assumptions and Dependencies

- In order to load map tiles and place data, it is assumed that users will have a reliable internet connection.
- It is estimated that at least 10 locations of interest within 25 km of Walasmulla may be documented with sufficiently accurate information (opening hours, descriptions, coordinates) within the project schedule.
- The system depends on the continuing availability and free-tier use limits of the chosen external Map API (Google Maps or Leaflet.js).

- It is expected that stakeholder requirements acquired from peer consultation (see Appendix A of the project proposal) remain representative of real tourist and administrator demands.

3. External Interface Requirements

3.1 User Interfaces

- Tourist Interface: a publicly accessible home page with a list of interesting locations, category filter controls, a search bar, a place detail page (description, opening hours, travel advice, distance, embedded map), and a visit-plan builder page that lists specific locations along with travel advice and an estimated time of arrival.
- Administrator Interface: a safe login site and a dashboard with forms and list views for controlling categories, inspecting and blocking registered tourist accounts, and adding, updating, and removing place records.
- The interface will include a unified navigation bar, distinct category icons, and standard form validation/error messages that are shown inline next to the pertinent field. It will also be mobile-responsive.

3.2 Hardware Interfaces

No specialist hardware is directly interfaced with by the system. All that is needed is a typical client device a desktop, laptop, tablet, or smartphone that can run a contemporary web browser and network hardware such as Wi-Fi or mobile data that is adequate to connect to the internet.

3.3 Software Interfaces

- Frontend: React.js, which uses HTML, CSS, and JavaScript to communicate with the backend over HTTP/REST API requests.
- Backend: Node.js with the Express.js framework, which exposes RESTful APIs for place management, visit-plan creation, and authentication.
- Database: MongoDB, which the backend uses to read and write place, category, and administrator/account data.
- External Map API: Leaflet.js or the Google Maps JavaScript API, which render locations and, where accessible, compute routes and distances.
- Development and testing tools: Postman for API testing, Visual Studio Code as the development environment, and Git/GitHub for version control.

3.4 Communications Interfaces

Using a RESTful JSON-based API, all communication between the client (browser) and the backend server takes place over HTTP/HTTPS. HTTPS is also used to send requests to the external Map API. The essential features of this system do not require email, messaging, or other communication methods.

4. System Features

4.1 Place Directory with Category Filtering

4.1.1 Description and Priority

Allows tourist users to browse the full list of documented places of interest and narrow the list by category (nature, heritage, religious, cultural) or by keyword search. Priority: High.

4.1.2 Stimulus/Response Sequences

- Tourist opens the home page → System displays the full list of places with names, categories, and distance.
- Tourist selects a category filter (or enters a search keyword) → System refreshes the list to show only matching places.

4.1.3 Functional Requirements

REQ-1.1: The system shall display a list of all active places of interest on the home page.

REQ-1.2: The system shall allow the user to filter the place list by one or more predefined categories.

REQ-1.3: The system shall allow the user to search places by keyword (place name).

REQ-1.4: The system shall display, for each place in the list, its name, category, and distance from Walasmulla.

REQ-1.5: The system shall display a clear message when no places match the selected filter or search term.

4.2 Detailed Place View with Map Integration

4.2.1 Description and Priority

Allows tourist users to view full details of an individual place, including description, opening hours, entry requirements, travel tips, and its location on an embedded map. Priority: High.

4.2.2 Stimulus/Response Sequences

- Tourist selects a place from the list → System displays the place detail page with description, opening hours, travel tips, and an embedded map marker for that location.

4.2.3 Functional Requirements

REQ-2.1: The system shall display, for each place, its description, category, opening hours, and estimated visit duration.

REQ-2.2: The system shall display each place's location as a marker on an embedded map using the integrated Map API.

REQ-2.3: The system shall display practical travel tips associated with the place, where available (e.g., dress code, parking availability).

REQ-2.4: The system shall display the distance of the place from Walasmulla town centre.

4.3 Day-Visit Itinerary Builder

4.3.1 Description and Priority

Allows tourist users to select multiple places from the directory and generate a simple, ordered one-day visit plan. Priority: High.

4.3.2 Stimulus/Response Sequences

- Tourist adds one or more places to their visit plan while browsing → System maintains a running list of selected places.
- Tourist requests to generate the plan → System compiles the selected places into an itinerary showing estimated time at each location and suggested visiting order.

4.3.3 Functional Requirements

REQ-3.1: The system shall allow a tourist user to add or remove places from a temporary visit-plan list.

REQ-3.2: The system shall generate a one-day itinerary listing the selected places along with their estimated visit duration.

REQ-3.3: The system shall warn the user if the total estimated time for the selected places is likely to exceed a reasonable single-day travel window.

REQ-3.4: The system should allow the user to view the selected places together on a single map for the day.

4.4 Administrator Content Management (CRUD)

4.4.1 Description and Priority

Allows an authenticated administrator to create, read, update, and delete place records, including managing categories and associated details. Priority: High.

4.4.2 Stimulus/Response Sequences

- Administrator logs in and navigates to the dashboard → System displays a list of all existing place records with edit/delete controls.
- Administrator submits a new place form or edits an existing record → System validates the input and saves the changes to the database.

4.4.3 Functional Requirements

REQ - 4.1: The system shall allow an authenticated administrator to add a new place record, including name, category, description, opening hours, travel tips, and location coordinates.

REQ - 4.2: The system shall allow an authenticated administrator to edit any existing place record.

REQ - 4.3: The system shall allow an authenticated administrator to delete a place record.

REQ - 4.4: The system shall validate required fields (name, category, and location) before saving a place record and shall display an error message if validation fails.

REQ - 4.5: The system shall allow an authenticated administrator to create new place categories.

REQ - 4.6: The system shall allow an authenticated administrator to update or delete existing place categories.

4.5 Administrator Authentication and User Management

4.5.1 Description and Priority

Enables administrators to securely log in, view, and manage registered tourist accounts. This includes the ability to block or remove accounts that misuse the system. Priority - Medium.

4.5.2 Stimulus/Response Sequences

- Administrator enters credentials on the login page → System authenticates the credentials and grants access to the dashboard, or displays an error if invalid.
- Administrator views the list of registered tourist accounts → System displays account status and provides an option to block or remove an account.

4.5.3 Functional Requirements

REQ-5.1: The system shall require administrators to log in with a valid username and password before accessing the dashboard.

REQ-5.2: The system shall lock or reject access after multiple consecutive failed login attempts, in line with standard security practice.

REQ-5.3: The system shall allow an administrator to view a list of registered tourist accounts.

REQ-5.4: The system shall allow an administrator to block or remove a tourist account found to be misusing the system.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Under typical network conditions, the place directory page should load and show the first list of locations in three seconds.
- Results from filtering or searching the place list must be updated in less than two seconds.
- In keeping with the small-scale, single-region nature of this project, the system must accommodate at least 20 concurrent users without appreciable response time degradation.

5.2 Safety Requirements

There is no direct risk to physical safety because the system does not control any physical equipment. In order to prevent visitors from acting in a way that is unsafe or culturally inappropriate when visiting sensitive destinations, the system must explicitly provide travel advice and site-specific standards (such as dress codes for religious sites).

5.3 Security Requirements

- Administrator passwords shall be stored using a secure, salted hashing algorithm and never stored in plain text.
- All authentication requests and any transmission of sensitive data shall occur over HTTPS.
- Administrator-only functions (add, edit, delete place; manage accounts) shall be protected by session- or token-based authentication and shall not be accessible to unauthenticated users.
- Input received from users (search terms, form fields) shall be validated and sanitised on the backend to prevent injection attacks.

5.4 Software Quality Attributes

- Usability: A first-time tourist should be able to locate a location and create an itinerary without the need for outside guidance, thanks to the Tourist Interface's simplicity and intuitiveness.
- Maintainability: To make future upgrades easier, the software will have distinct modules for Place Management, Authentication, and Visit Plan Generator.
- Reliability: The system must gracefully and without crashing handle invalid input and API errors (such as the unavailability of the map service).
- Portability: the responsive frontend must work well on both desktop and mobile screen sizes as well as the most recent versions of the major browsers.

5.5 Business Rules

- Place records and visitor accounts can only be created, edited, or deleted by authenticated administrators.
- For a place record to be published in the tourist-facing directory, it must fall under at least one legitimate category.
- Registration is only necessary to store a plan for future access; it is not necessary for tourists to browse locations or create a temporary visitation plan.

6. Other Requirements

Place, category, and account data must be stored in a MongoDB database with a schema that facilitates effective category and distance filtering. Internationalization: Version 1.0 does not require multilingual support (such as Sinhala), but it is a potential future improvement. The first release focuses solely on English-language content. Legal requirements: the system must adhere to established academic data-handling procedures for this research, and no personally identifiable tourist information other than account credentials (if registration is used) may be gathered.

Appendix A: Glossary

Tourist User: A visitor accessing the system to browse places of interest and build a day-visit plan.

Administrator: An authorised system manager who maintains place records and tourist accounts.

Place of Interest: A documented tourist attraction (nature, heritage, religious, or cultural site) within 25 km of Walasmulla.

Visit Plan / Itinerary: An ordered list of selected places, with estimated visit durations, generated for a single day's travel.

CRUD: Create, Read, Update, Delete - the four basic operations available to an administrator on place records.

MERN Stack: MongoDB, Express.js, React.js, and Node.js - the technology stack used to build the system.

REST API: A web service interface following Representational State Transfer principles, used for communication between the frontend and backend.

Appendix B: Analysis Models

The three-tier architecture of the system consists of a Data Tier (MongoDB collections for Admin Data and Places & Categories), an Application Tier (Node.js/Express.js REST API with Authentication, Place Management, and Visit Plan Generator modules), and a Client Tier (Tourist and Administrator Interfaces built in React.js). The Client Tier sends independent map requests to an external Map API (such as Google Maps or Leaflet.js) and interacts with the Application Tier over HTTP/REST. The approved project proposal's Figure 1 (a high-level block diagram of the suggested system) and this design match.

Table 2 below reproduces the preliminary list of places identified during requirements elicitation, which forms the initial dataset for the Place Management module.

#	Place Name	Category	Distance from Walasmulla
1	Mulkirigala Raja Maha Vihara	Religious/Heritage	5 km
2	Tangalle Beach	Nature	20 km
3	Wewurukannala Vihara	Religious	22 km
4	Rekawa Turtle Conservation	Nature	25 km
5	Hummanaya Blowhole	Nature	24 km
6	Kasagala Raja Maha Vihara	Religious/Heritage	15 km
7	Muruthawela Reservoir	Nature	10 km
8	Kalametiya Bird Sanctuary	Nature	25 km
9	Parewella Natural Swimming Area	Nature	20 km
10	Ussangoda National Park	Nature	25 km

Table 2: Preliminary List of Places of Interest (within 25 km of Walasmulla)

The 12-week project plan, covering the Requirements, System Design, Development, Testing, and Finalisation phases, is documented in the approved project proposal (Appendix B, Table 3: Detailed Project Development Timeline by Phase) and is adopted as the delivery schedule for this SRS.

Table 3: Detailed Project Development Timeline by Phase

No.	Main Activities	M1	M2	M3	M4	M5	M6	M7	M8	M9	Period
1	Problem identification and preliminary research	X									M1
2	Literature review and existing tourism system analysis	X	X								M1-2
3	Stakeholder identification and requirements gathering		X								M2
4	Requirements analysis and SRS documentation		X	X							M2-3
5	Database schema, API architecture, and system design			X							M3
6	UI/UX design and prototype preparation (Figma)			X	X						M3-4
7	Backend setup, Authentication & Admin profile dev				X						M4
8	Place Management Module (Admin CRUD operations)				X	X					M4-5
9	Frontend dev: Place Directory & Category Filtering					X					M5
10	Map integration (Google Maps/Leaflet) & logic					X	X				M5-6
11	Detailed place view and travel tips implementation						X				M6
12	Day-Visit Itinerary Builder algorithm & development						X	X			M6-7
13	Itinerary builder frontend & map visualization							X			M7
14	System integration and backend security improvements							X	X		M7-8
15	Functional, API, and usability testing								X		M8
16	User evaluation and bug fixing/system refinements								X	X	M8-9
17	Final documentation and project demonstration prep									X	M9

Appendix C: To Be Determined List

1. TBD-1: Final choice between Google Maps JavaScript API and Leaflet.js, pending evaluation of free-tier usage limits.
2. TBD-2: Exact algorithm for ordering places within a generated day-visit plan (e.g., by distance, by category, or by user-defined order).
3. TBD-3: Whether tourist account registration will be mandatory for saving a visit plan, or whether an anonymous/session-based option will also be supported.
4. TBD-4: Final list and wording of site-specific guidelines (e.g., dress codes) to be sourced and verified for each religious/cultural site.